DETAILED PROJECT REPORT: GIT & GITHUB WORKFLOW

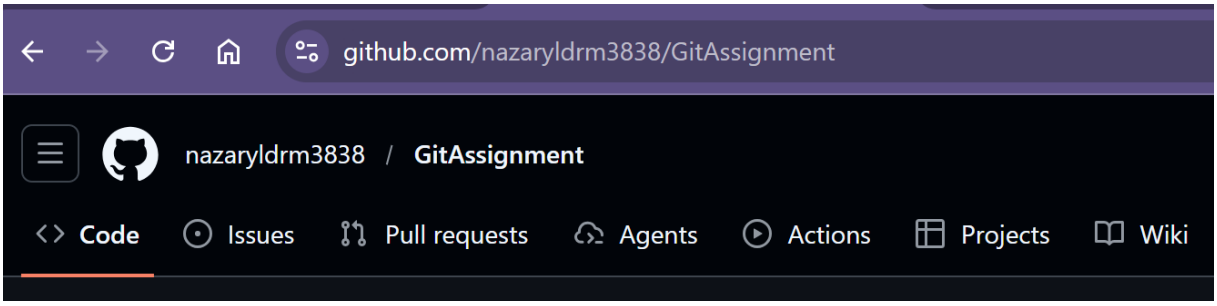
Course: Software Development Tools

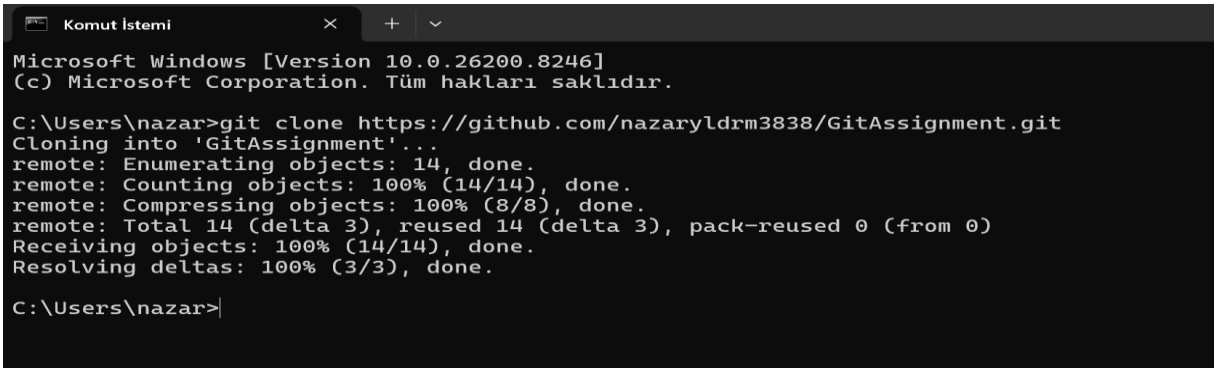
Name: Nazar Yıldırım

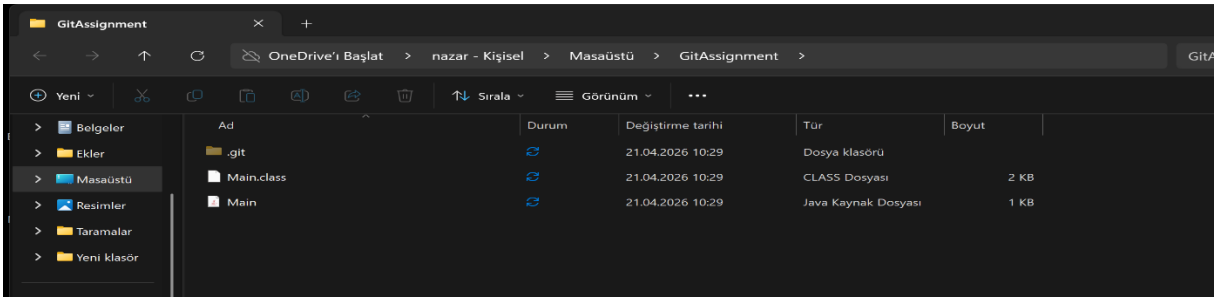
Repository: https://github.com/nazaryldrm3838/GitAssignment

PART 1: Repository Setup and Initial Development (Steps a - j)

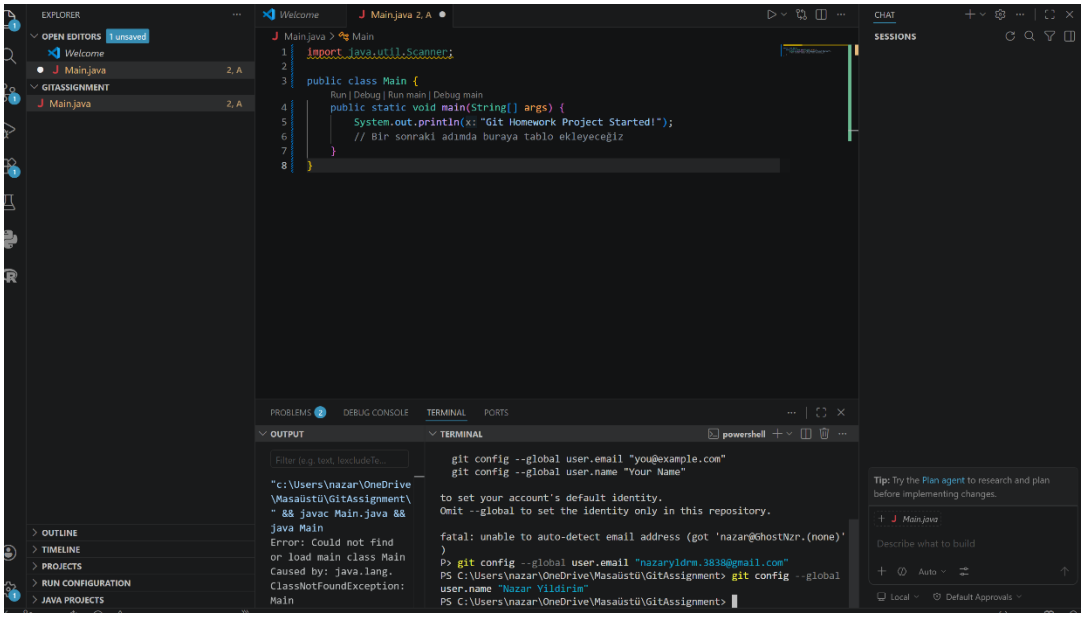
- **Task a, b, c (Initialization):** The project began by creating a directory named GitAssignment. I initialized a local Git repository using the git init command and created a Java class named Main.java.



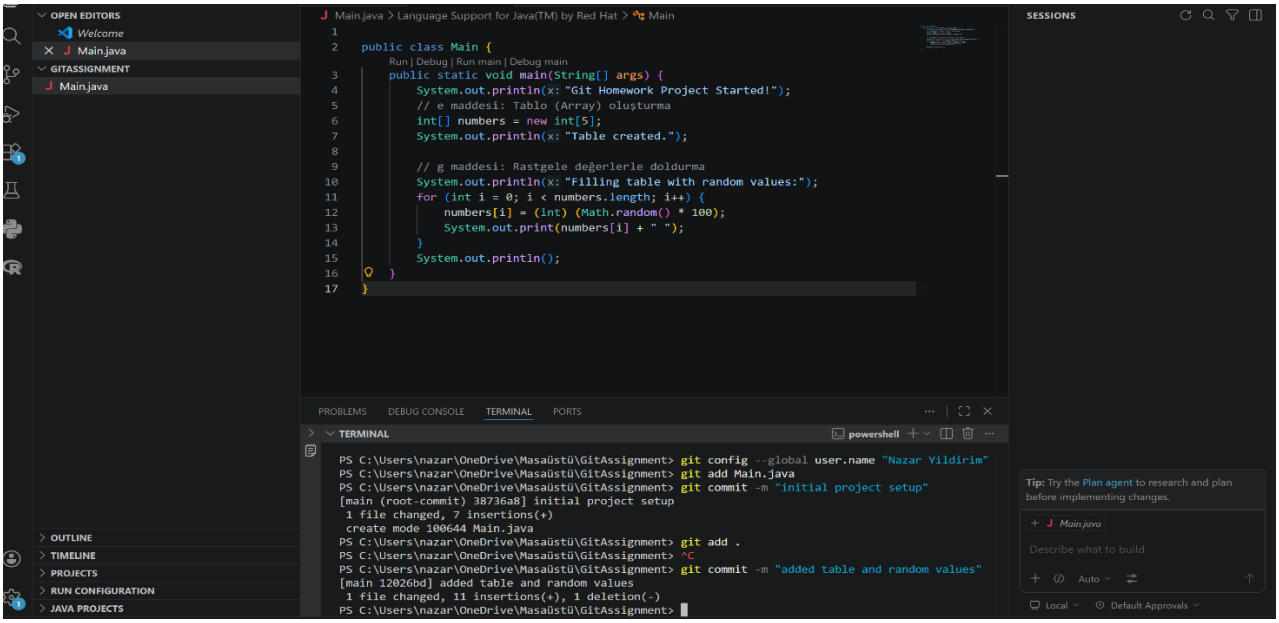




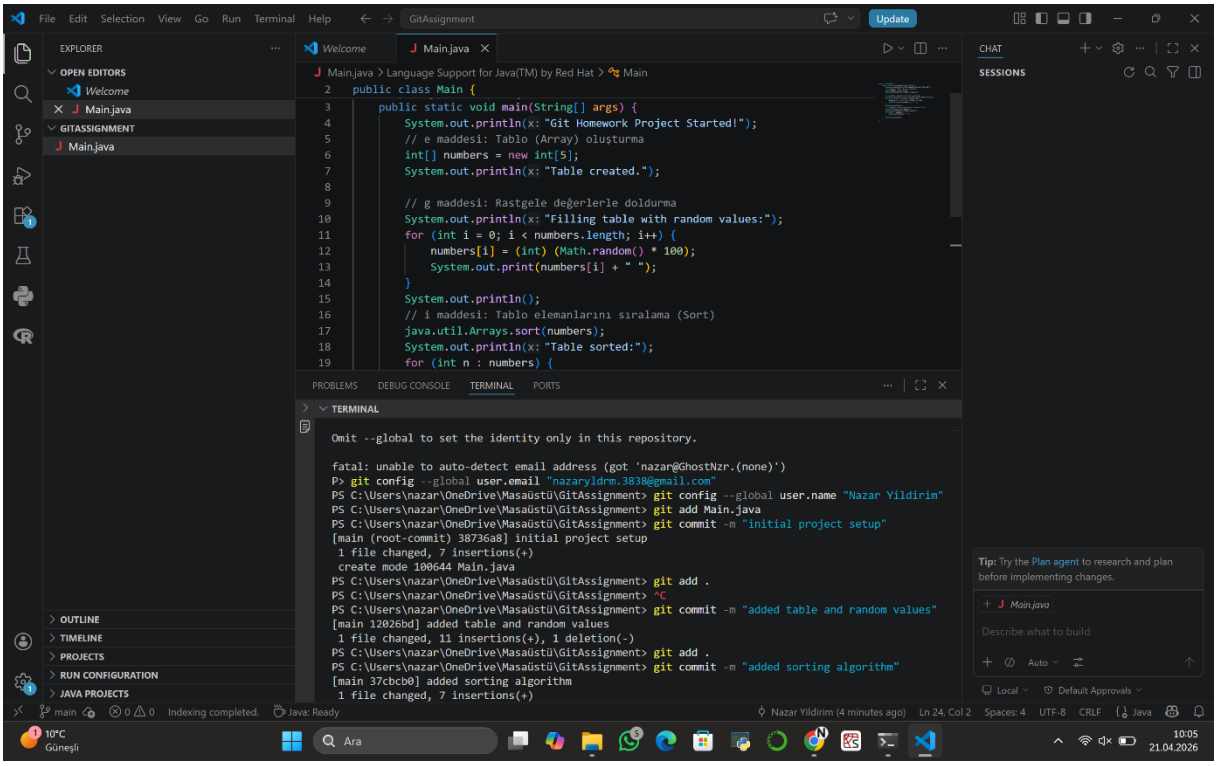
- **Task d (First Commit):** After setting up the basic structure, I performed the first commit with the message *"initial project setup"*.



- **Task e, g (Feature Implementation):** I implemented the core logic of the application: creating an integer array and populating it with random values using Math.random().
- **Task f, h (Second Commit):** Once the array logic was functional, I staged the changes and committed them with the message *"added table and random values"*.

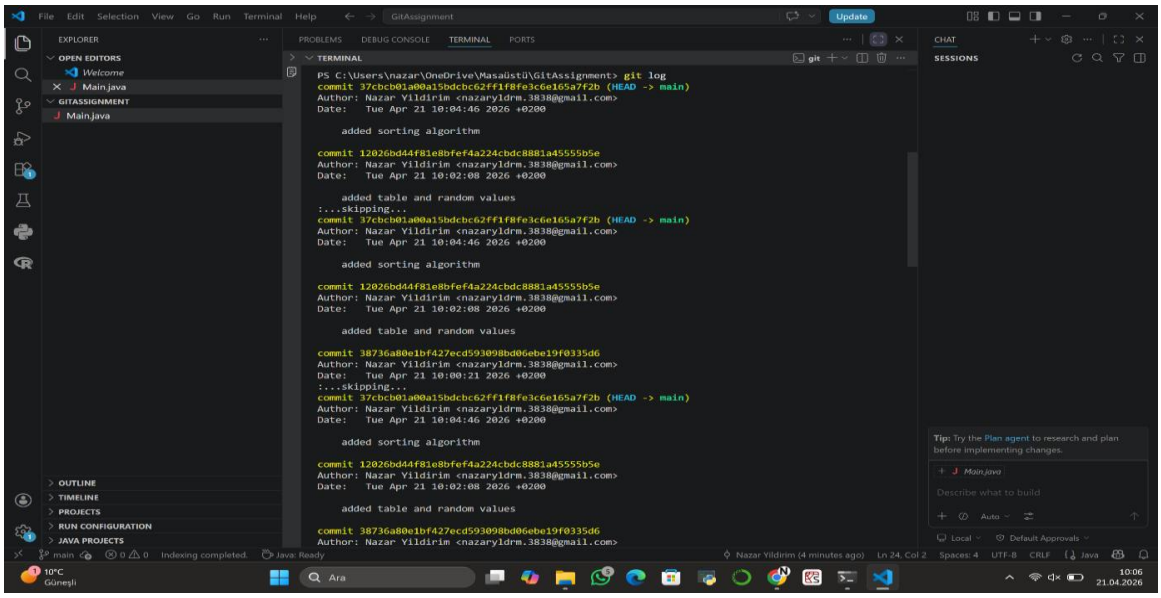


- **Task i, j (Final Logic & Third Commit):** I added a sorting algorithm using java.util.Arrays.sort(). To finalize the local development phase, I made a third commit: *"added sorting algorithm"*.

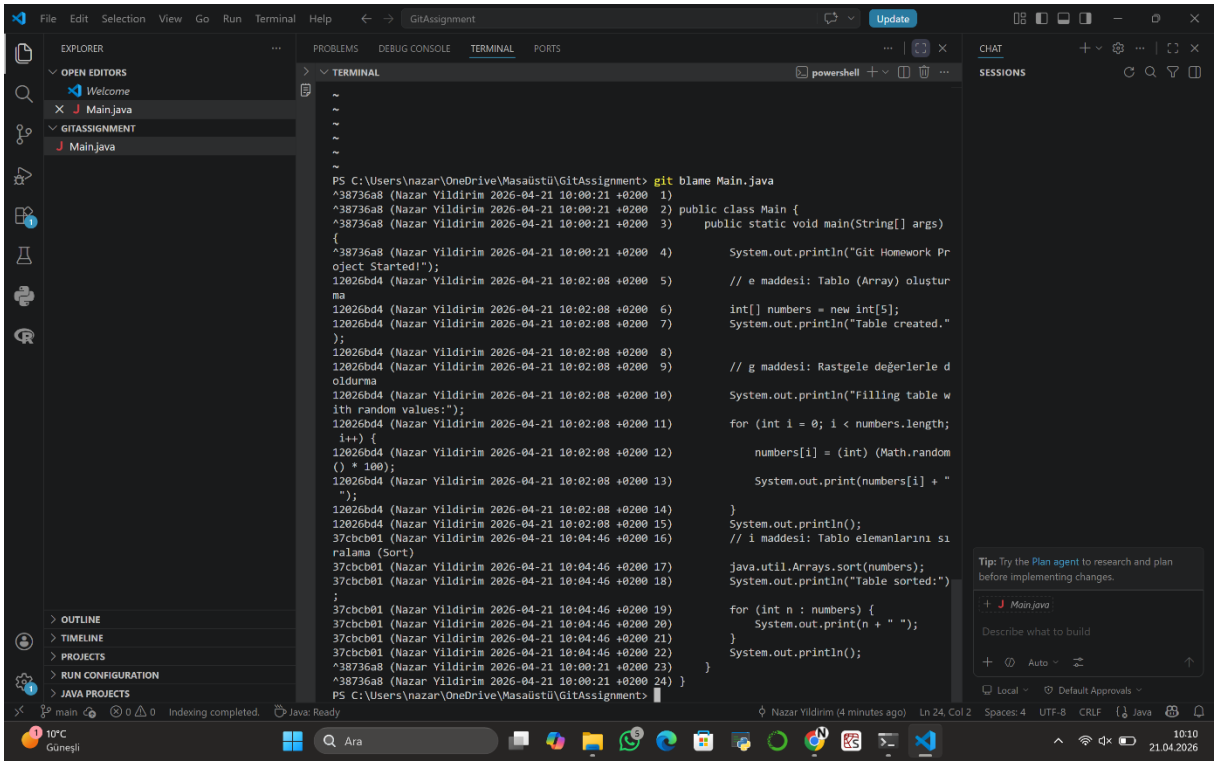


PART 2: History Inspection and Version Control (Steps k - o)

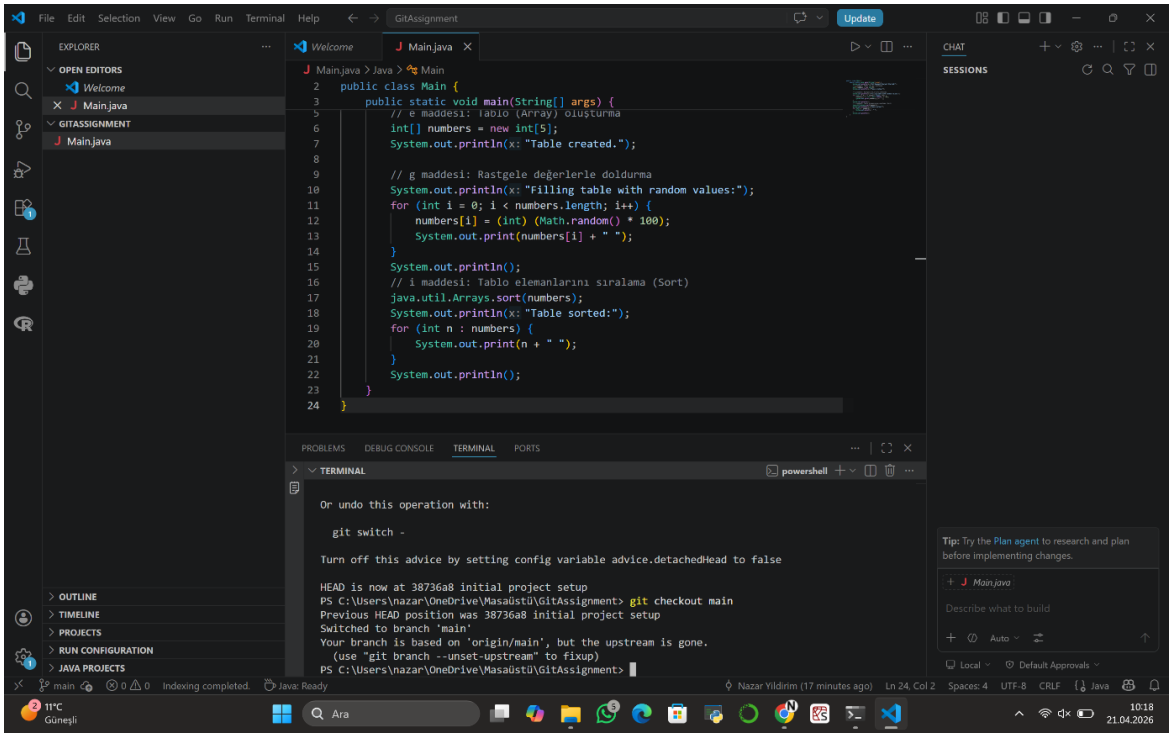
- **Task k (Git Log):** To verify the project's evolution, I used git log. bu komut tüm commit geçmişini, kimin ne zaman yaptığını ve benzersiz ID'leri (hash) görmemi sağladı.



- **Task l (Git Blame):** I executed git blame Main.java to inspect authorship. This tool displayed which developer was responsible for each specific line of code.



- **Task m (Git Checkout):** I practiced "time travel" by using git checkout [commit-id]. This allowed me to view the project's state at an earlier stage. I then returned to the current version using git checkout main.



- **Task n, o (Error Recovery):** I simulated an accidental deletion in the source code. By using git restore Main.java, I successfully recovered the lost data from the last committed state, demonstrating Git's reliability in data protection.(In this step i deleted the 20th line and i restore it.)

```
17            java.util.Arrays.sort(numbers);
18            System.out.println(x: "Table sorted:");
19            for (int n : numbers) {
20            }
21            System.out.println();
22        }
23    }
```

```
19            for (int n : numbers) {
20                System.out.print(n + " ");
21            }
22            System.out.println();
23        }
24    }
```

PROBLEMS DEBUG CONSOLE TERMINAL PORTS

TERMINAL powershell

```
ralama (Sort)
37cbcb01 (Nazar Yildirim 2026-04-21 10:04:46 +0200 17)        java.util.Arrays.sort(numbers);
37cbcb01 (Nazar Yildirim 2026-04-21 10:04:46 +0200 18)        System.out.println("Table sorted:")
;
37cbcb01 (Nazar Yildirim 2026-04-21 10:04:46 +0200 19)        for (int n : numbers) {
37cbcb01 (Nazar Yildirim 2026-04-21 10:04:46 +0200 20)            System.out.print(n + " ");
37cbcb01 (Nazar Yildirim 2026-04-21 10:04:46 +0200 21)        }
37cbcb01 (Nazar Yildirim 2026-04-21 10:04:46 +0200 22)        System.out.println();
^38736a8 (Nazar Yildirim 2026-04-21 10:00:21 +0200 23)    }
^38736a8 (Nazar Yildirim 2026-04-21 10:00:21 +0200 24) }
PS C:\Users\nazar\OneDrive\Masaüstü\GitAssignment> git restore Main.java
```

PART 3: GitHub Integration and Remote Operations (Steps p - s)

- **Task p (Remote Push):** I created a public repository on GitHub and linked it to my local machine. I successfully pushed all local commits to the cloud using git push origin main.

TERMINAL powershell

```
Your branch is based on 'origin/main', but the upstream is gone.
  (use "git branch --unset-upstream" to fixup)
PS C:\Users\nazar\OneDrive\Masaüstü\GitAssignment> git push origin main
info: please complete authentication in your browser...
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 1.10 KiB | 225.00 KiB/s, done.
Total 9 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/nazaryldrm3838/GitAssignment.git
 * [new branch]      main -> main
```

- **Task r, s (Clone & Recovery):** To test the remote backup, I deleted the local project folder. I then used git clone [URL] to download the entire project back to my computer, proving that the project is safely backed up in the cloud.

```
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\nazar\OneDrive\Masaüstü> git clone https://github.com/nazaryldrm3838/GitAssignment.git
Cloning into 'GitAssignment'...
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 9 (delta 2), reused 9 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (9/9), done.
Resolving deltas: 100% (2/2), done.
PS C:\Users\nazar\OneDrive\Masaüstü>
```

PART 4: Release Management and Tagging (Task t)

- **Task t (Tagging):** Versioning is essential for software releases. I used git tag -a v1.0 -m "Final Version" to mark the current stable state. This tag was then pushed to GitHub to provide a formal release point for the assignment.

```
PS C:\Users\nazar\OneDrive\Masaüstü\GitAssignment> git tag -a v1.0 -m "Final Version"
PS C:\Users\nazar\OneDrive\Masaüstü\GitAssignment> git push origin v1.0
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 173 bytes | 173.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/nazaryldrm3838/GitAssignment.git
 * [new tag]         v1.0 -> v1.0
PS C:\Users\nazar\OneDrive\Masaüstü\GitAssignment>
```

PART 5: Branching Strategy and Merging (Steps u - y)

- **Task u, w (Branching):** I followed a modern development workflow by creating a new branch named feature-sorting-fix. This isolated my new changes from the stable main code.

```
PROBLEMS   DEBUG CONSOLE   TERMINAL   PORTS
TERMINAL                                   powershell
PS C:\Users\nazar\OneDrive\Masaüstü\GitAssignment> git checkout -b feature-sorting-fix
Switched to a new branch 'feature-sorting-fix'
```

- **Task x (Improvements):** While in the new branch, I modified the code to include a confirmation message about the sorting process.

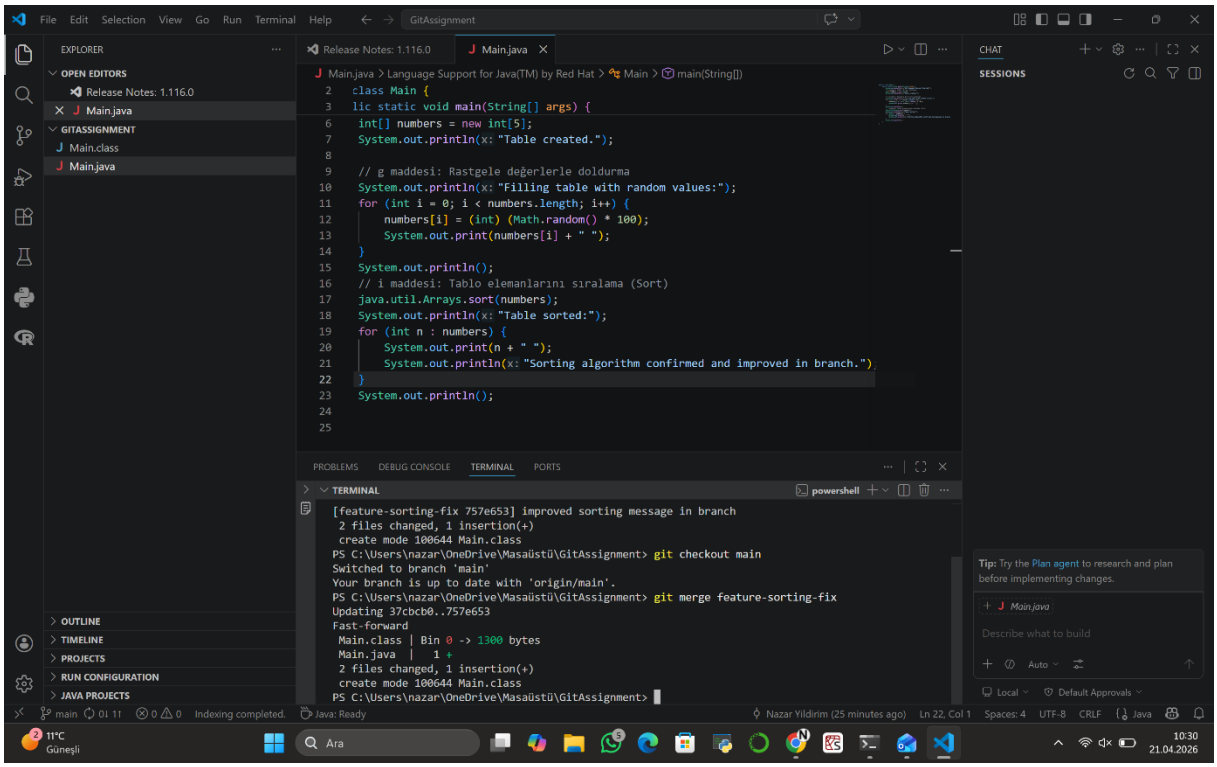
```java
    System.out.println();
    // i maddesi: Tablo elemanlarını sıralama (Sort)
    java.util.Arrays.sort(numbers);
    System.out.println(x: "Table sorted:");
    for (int n : numbers) {
        System.out.print(n + " ");
        System.out.println(x: "Sorting algorithm confirmed and improved in branch.");
    }
    System.out.println();
```

PROBLEMS DEBUG CONSOLE TERMINAL PORTS

TERMINAL powershell

```
PS C:\Users\nazar\OneDrive\Masaüstü\GitAssignment> git checkout -b feature-sorting-fix
Switched to a new branch 'feature-sorting-fix'
PS C:\Users\nazar\OneDrive\Masaüstü\GitAssignment> git add .
PS C:\Users\nazar\OneDrive\Masaüstü\GitAssignment> git commit -m "improved sorting message in bran
ch"
[feature-sorting-fix 757e653] improved sorting message in branch
 2 files changed, 1 insertion(+)
 create mode 100644 Main.class
PS C:\Users\nazar\OneDrive\Masaüstü\GitAssignment>
```

- **Task y (Merging):** After testing the changes, I switched back to the main branch and used git merge to integrate the improvements. This ensured a safe and organized way to update the project.



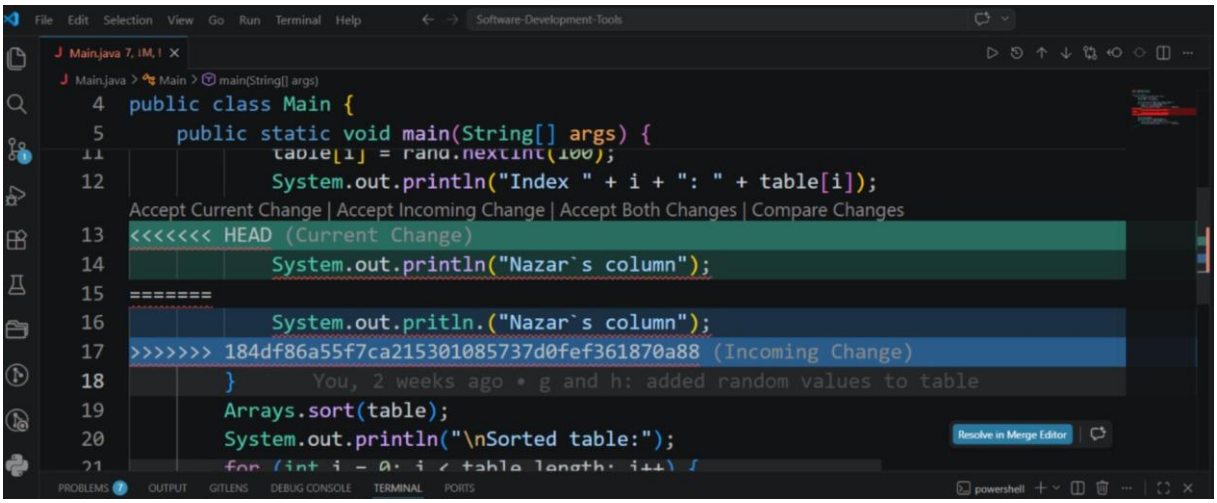
PART6: Collaboration and Merge Conflict Resolution (Step z)

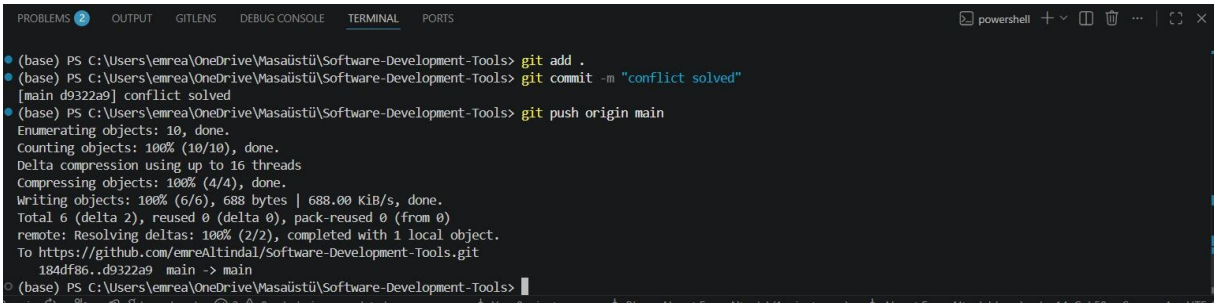
In this final phase, I collaborated with a classmate to simulate a real-world teamwork scenario. The objective was to create and resolve a "merge conflict," which occurs when two developers modify the same line of code simultaneously.

- **Step z1 (Generating the Conflict):** While my classmate was working on the local repository, I accessed the GitHub web interface and modified line 13 of Main.java. At the same time, my classmate made a different modification to the same line in their local environment. When my classmate attempted to git push their changes, Git rejected the operation because the remote version was ahead of the local one.

```
9           System.out.println("Table with random values:");
10          for (int i = 0; i < table.length; i++) {
11              table[i] = rand.nextInt(100);
12              System.out.println("Index " + i + ": " + table[i]);
13              System.out.println("Nazar`s column");
14              System.out.pritln("Nazar`s column");
15          }
16          Arrays.sort(table);
17          System.out.println("\nSorted table:");
18          for (int i = 0; i < table.length; i++) {
```

- **Step z2 (Resolving the Conflict):** To resolve this, my classmate performed a git pull. This action triggered a merge conflict, highlighting the differences between our codes in Visual Studio Code. We carefully reviewed both changes and decided to use the **"Accept Both Changes"** option to merge our work. After resolving the conflict manually, we performed a final add, commit, and push to synchronize the repository.





Final code was like this.

Code Blame 23 lines (21 loc) · 784 Bytes

```java
import java.util.Arrays;
import java.util.Random;

public class Main {
    public static void main(String[] args) {
        int[] table = new int[5];
        Random rand = new Random();

        System.out.println("Table with random values:");
        for (int i = 0; i < table.length; i++) {
            table[i] = rand.nextInt(100);
            System.out.println("Index " + i + ": " + table[i]);
            System.out.println("Nazar`s column");
            System.out.pritln("Nazar`s column");
        }
        Arrays.sort(table);
        System.out.println("\nSorted table:");
        for (int i = 0; i < table.length; i++) {
            System.out.println("!!!!!!!!!!!!!!!!!!!!!!!!!!!!");
            System.out.println("Index " + i + ": " + table[i]);
        }
    }
}
```

CONCLUSION

This assignment successfully covered the full lifecycle of a software project using Git and GitHub. I have mastered essential skills such as version tracking, history auditing, remote synchronization, and branching strategies, which are fundamental for professional software engineering.